

PROYECTO 3 — MIGRACIÓN CLOUD

Metodologías de Desarrollo y Despliegue de Aplicaciones para Ciencia de Datos

ABC Analytics

Migración Cloud de la API de Predicción de Ventas

De infraestructura on-premise a Microsoft Azure

Equipo:

Juan Manuel Campos Enrique / Mónica Penacho

Profesor: Federico Muñoz Babiano

Universidad Internacional de Valencia (VIU)

Mayo 2026

Índice de Contenidos

Índice de Contenidos	2
1. Resumen Ejecutivo	3
2. Descripción del Problema y Caso de Uso	4
2.1 Situación inicial: infraestructura on-premise.....	4
2.2 Valor de la migración a la nube	4
3. Arquitectura de la Solución	6
3.1 Descripción general.....	6
3.2 Flujo end-to-end	6
3.3 Justificación de tecnologías	6
4. Implementación del Pipeline	7
4.1 API con FastAPI	7
4.2 Containerización con Docker	7
4.3 Pipeline CI/CD con GitHub Actions.....	8
Job 1: lint-and-test (Integración Continua).....	8
Job 2: build-and-push (Despliegue Continuo)	8
4.4 Infraestructura como Código con Terraform	9
5. Infraestructura y Despliegue en Azure	9
5.1 Recursos desplegados	9
5.2 Configuración del Container Instance	9
5.3 Proceso de despliegue con Terraform	9
5.4 Gestión de credenciales	10
6. Metodología Ágil (Scrum).....	11
6.1 Organización del trabajo.....	11
6.2 Sprints del proyecto.....	11
6.3 User Stories principales.....	11
6.4 Retrospectiva del proyecto	12
7. Resultados Obtenidos.....	13
7.1 Métricas del proyecto.....	13
7.2 API desplegada en producción	13
8. Problemas Encontrados y Soluciones	14
9. Conclusiones	15
10. Trabajo Futuro	16
10.1 Observabilidad y monitorización	16
10.2 Mejoras en el pipeline.....	16
10.3 Evolución del modelo de predicción.....	16
10.4 Escalabilidad y alta disponibilidad.....	16
Referencias.....	17

1. Resumen Ejecutivo

El presente documento describe la migración completa de la API de predicción de ventas de ABC Analytics desde una infraestructura on-premise hacia Microsoft Azure, aplicando prácticas modernas de DevOps: containerización con Docker, pipeline de integración y despliegue continuo (CI/CD) con GitHub Actions, e Infraestructura como Código (IaC) con Terraform.

La situación de partida era una API de predicción de ventas desplegada manualmente en servidores físicos, sin automatización, sin reproducibilidad del entorno y con altos costes operativos. Cada despliegue requería intervención manual, lo que aumentaba el riesgo de errores y dificultaba la escalabilidad del sistema.

La solución implementada resuelve estos problemas mediante una arquitectura cloud nativa que automatiza por completo el ciclo de vida del software: desde el commit del desarrollador hasta el despliegue en producción en Azure Container Instances (ACI), pasando por validación automática de calidad y construcción de la imagen Docker.

Aspecto	Antes (On-Premise)	Después (Azure Cloud)
Despliegue	Manual, propenso a errores	☑ Automatizado con GitHub Actions
Reproducibilidad	Depende del entorno local	☑ Docker garantiza entorno idéntico
Tiempo de despliegue	Horas (manual)	☑ Minutos (pipeline automatizado)
Infraestructura	Servidores físicos fijos	☑ Definida en código (Terraform)
Escalabilidad	Sin escalar sin hardware	☑ Ajustable en Azure
Coste	CAPEX elevado y fijo	☑ ~8.50€/mes (pay-per-use)

El resultado final es una API desplegada en producción en Azure (región Spain Central) con acceso público, documentación automática Swagger, pipeline CI/CD operativo y toda la infraestructura reproducible en menos de 90 segundos con un único comando.

2. Descripción del Problema y Caso de Uso

2.1 Situación inicial: infraestructura on-premise

ABC Analytics es una empresa de análisis de datos que opera una API de predicción de ventas utilizada internamente por su equipo comercial. Esta API, desarrollada en Python, permite a los equipos de ventas obtener previsiones de demanda para sus productos principales, introduciendo parámetros como el identificador del producto, las ventas base del período anterior y el mes objetivo.

Sin embargo, la infraestructura en la que se ejecuta esta API presenta limitaciones críticas que dificultan su operación y evolución:

- Despliegue completamente manual: cualquier actualización del código requería acceso directo al servidor, ejecución de comandos manuales y verificación ad hoc. Este proceso tardaba horas y dependía de la disponibilidad de personal técnico.
- Ausencia de automatización: no existía ningún mecanismo de validación automática del código antes de desplegarlo. Los errores solo se detectaban en producción, con el impacto correspondiente en el servicio.
- Entorno no reproducible: la API funcionaba gracias a una configuración específica del servidor que no estaba documentada. Si el servidor fallaba o había que migrar a otro, el proceso de restauración era lento e impredecible.
- Escalabilidad limitada: al tratarse de infraestructura física, aumentar la capacidad requería adquirir nuevo hardware, con los costes y tiempos de aprovisionamiento que eso implica.
- Coste fijo elevado: los servidores físicos generan coste independientemente del uso real del sistema, lo que resulta ineficiente en períodos de baja demanda.

2.2 Valor de la migración a la nube

Nuestra apli cae en Replatform según el motor 6R: complejidad media + equipo con conocimiento medio cloud → equilibrio óptimo esfuerzo/beneficio.

La migración a Azure resuelve todas las limitaciones anteriores mediante tres pilares tecnológicos:

Automatización mediante CI/CD: cada vez que un desarrollador sube código al repositorio, el pipeline de GitHub Actions valida automáticamente la calidad del código, ejecuta los test y, si todo es correcto, construye y despliega la nueva versión sin intervención humana.

Portabilidad mediante Docker: la aplicación se empaqueta en un contenedor que incluye todas sus dependencias. Este contenedor funciona de forma idéntica en el portátil del desarrollador, en el entorno de CI/CD y en Azure, eliminando el clásico problema de "funciona en mi máquina".

Reproducibilidad mediante IaC: toda la infraestructura Azure (grupos de recursos, registros de contenedores, instancias de contenedor) está definida en código Terraform. Esto significa que el entorno completo puede recrearse desde cero en menos de 90 segundos con un único comando.

2.3 Análisis 6R y Wave Planning

Previo al diseño de la solución, se realizó un análisis formal del portafolio de aplicaciones on-premise de ABC Analytics aplicando el marco de las 6 R's de migración (Gartner/AWS). El análisis completo, incluyendo el motor de clasificación implementado en Python, los resultados por aplicación, el diseño de arquitectura cloud objetivo y el Wave Planning, está documentado en el notebook del repositorio:

https://github.com/monicapenacho/sales-prediction-api-migration/blob/main/portfolio/Portfolio_ABC_Analysis.ipynb

Portafolio analizado: ABC Analytics opera 5 aplicaciones on-premise. El motor de clasificación aplica las reglas de las 6 R's a cada aplicación en función de su criticidad, complejidad técnica, uso mensual, presencia de datos sensibles y experiencia cloud del equipo. Los resultados son:

- **Sales Prediction API → Replatform:** complejidad media + equipo con conocimiento medio cloud. Equilibrio óptimo esfuerzo/beneficio. **Es la app que se implementa en este proyecto.**
- **Dashboard KPIs Retail → Repurchase:** existe SaaS equivalente (Power BI Service). Migrar a SaaS elimina toda gestión de infraestructura. Wave 1.
- **Reportes legacy clientes → Retire:** alta complejidad + uso mínimo (4 usuarios/mes). No justifica el esfuerzo de migración. Wave 1.
- **ETL nocturno de ventas → Rehost:** Lift & Shift a Azure VM + Postgres flexible. Migrar sin cambiar código; el equipo aprende la nube con riesgo controlado. Wave 2.
- **Modelo scoring de clientes → Retain (por ahora):** criticidad muy alta + datos sensibles + equipo sin experiencia cloud suficiente. Migración diferida a Wave 3 cuando el equipo ya tiene práctica probada. Objetivo a largo plazo: Refactor sobre Azure ML.

Wave Planning: la migración se organiza en 3 oleadas que minimizan el riesgo y maximizan el aprendizaje del equipo:

- **Wave 1 — Quick wins (2 semanas):** Sales Prediction API (Replatform), Dashboard (Repurchase), Reportes legacy (Retire). El equipo aprende Docker, ACR, ACI y Terraform con apps de bajo riesgo.
- **Wave 2 — Datos (3 semanas):** ETL nocturno (Rehost a Azure VM). Requiere validar pipelines de datos y conectividad a base de datos en la nube.
- **Wave 3 — Crítico (4 semanas):** Modelo scoring de clientes (Retain → Refactor sobre Azure ML). Solo cuando el equipo tiene experiencia probada con las waves anteriores.

El análisis detallado, incluyendo el motor de clasificación Python, los diagramas Mermaid de arquitectura cloud, las métricas DORA esperadas tras Wave 1 y los planes de rollback por wave, está disponible en el notebook del repositorio (véase enlace anterior).

3. Arquitectura de la Solución

3.1 Descripción general

La arquitectura de la solución sigue el patrón GitOps: el repositorio de GitHub actúa como única fuente de verdad para el código, la configuración del pipeline y la definición de la infraestructura. Todo cambio en el sistema pasa obligatoriamente por este repositorio y por el pipeline automatizado.

Los componentes principales de la arquitectura son:

- **GitHub Repository:** almacena el código fuente de la API, los workflows de GitHub Actions y los archivos de Terraform. Es el punto de entrada de cualquier cambio en el sistema.
- **GitHub Actions (CI/CD):** orquesta el ciclo de integración y despliegue continuo. Se compone de dos jobs independientes con responsabilidades diferenciadas.
- **Azure Container Registry (ACR):** registro privado de imágenes Docker en Azure. Actúa como almacén de artefactos: recibe las imágenes construidas por el pipeline y las sirve a Azure Container Instances.
- **Azure Container Instances (ACI):** servicio de ejecución de contenedores en Azure sin necesidad de gestionar máquinas virtuales. Ejecuta la imagen Docker de la API con una IP pública y un FQDN accesible externamente.
- **Terraform:** herramienta de IaC que define y provisiona toda la infraestructura Azure de forma declarativa y reproducible.

3.2 Flujo end-to-end

El flujo completo desde el desarrollo hasta la disponibilidad en producción es el siguiente:

```
Developer → git push → GitHub Repository
GitHub Repository → trigger → GitHub Actions (CI: lint + tests)
CI aprobado → GitHub Actions (CD: docker build → push ACR)
ACR → imagen disponible → Terraform apply → Azure ACI
Azure ACI → API pública → Usuario final (http://<fqdn>:8000)
```

Este flujo garantiza que ningún cambio de código llega a producción sin haber superado las validaciones automáticas de calidad. La separación en dos jobs (CI y CD) es una decisión de diseño deliberada: el job de CI se ejecuta en todas las ramas del repositorio para detectar errores cuanto antes, mientras que el job de CD solo se activa en la rama principal y únicamente si CI ha pasado con éxito.

3.3 Justificación de tecnologías

Tecnología	Alternativas consideradas	Justificación de la elección
FastAPI	Flask, Django REST	Más rendimiento que Flask; Swagger automático incluido; validación de tipos con Pydantic sin configuración adicional
Docker	Despliegue directo en VM	Garantiza entorno idéntico en todos los contextos; estándar del sector para microservicios

Tecnología	Alternativas consideradas	Justificación de la elección
GitHub Actions	Jenkins, GitLab CI	Integración nativa con el repositorio GitHub; no requiere infraestructura adicional; trabajado en clase
Terraform	ARM Templates, Bicep, portal Azure manual	Multi-cloud; declarativo; idempotente; permite recrear entornos completos; visto en clase
Azure	AWS, Google Cloud	Plataforma trabajada durante el curso; Azure Students ofrece 100€ de crédito con correo universitario sin tarjeta; AWS Educate no proporciona acceso a servicios reales de infraestructura

4. Implementación del Pipeline

4.1 API con FastAPI

La API de predicción de ventas se ha implementado con FastAPI, un framework moderno de Python que combina alto rendimiento con facilidad de desarrollo. La aplicación expone tres endpoints:

Endpoint	Método	Descripción
/	GET	Confirma que la API está en funcionamiento y devuelve la versión actual
/health	GET	Health check para monitorización del contenedor en Azure; devuelve {"status": "ok"}
/predict	POST	Recibe datos de un producto y devuelve la predicción de ventas con nivel de confianza

El endpoint `/predict` recibe un objeto JSON con tres campos: el identificador del producto (`product`), las ventas base del período anterior (`base_sales`) y el mes objetivo (`month`). Aplica un factor de crecimiento estocástico entre -10% y +30% sobre las ventas base y devuelve la predicción junto con un nivel de confianza entre 0.75 y 0.95.

Los modelos de datos de entrada y salida se definen con Pydantic, lo que proporciona validación automática de tipos y genera documentación Swagger sin configuración adicional:

```
class SalesPrediction(BaseModel):
    product: str          # Identificador del producto
    base_sales: float      # Ventas base del período anterior
    month: int             # Mes objetivo para la predicción

class PredictionResponse(BaseModel):
    product: str
    predicted_sales: float  # Predicción resultante
    confidence: float       # Nivel de confianza (0-1)
```

4.2 Containerización con Docker

La containerización es el elemento que garantiza la reproducibilidad del entorno de ejecución. El Dockerfile de la API sigue las mejores prácticas de construcción de imágenes:

- Imagen base ligera: se utiliza python:3.11-slim para minimizar el tamaño de la imagen y reducir la superficie de ataque potencial.
- Separación de capas: las dependencias se instalan en una capa separada del código fuente, lo que permite reutilizar la caché de Docker en builds sucesivos cuando solo cambia el código.
- Health check integrado: el Dockerfile incluye una instrucción HEALTHCHECK que verifica periódicamente que el endpoint /health responde correctamente, permitiendo a Azure detectar y reiniciar automáticamente contenedores no saludables.
- Configuración por variables de entorno: la versión de la aplicación se inyecta mediante variables de entorno, siguiendo el principio de configuración externa de los doce factores (12-factor app).

La imagen resultante puede ejecutarse localmente con Docker Compose para desarrollo y pruebas, y se despliega en Azure Container Instances en producción sin ninguna modificación.

4.3 Pipeline CI/CD con GitHub Actions

El pipeline CI valida cada PR con lint + tests + cobertura $\geq 70\%$, y la imagen Docker se publica en nuestro propio ACR. Terraform se encarga de desplegarla en Azure Container Instances.

El pipeline de GitHub Actions se divide en dos jobs con responsabilidades claramente diferenciadas:

Job 1: lint-and-test (Integración Continua)

Este job se ejecuta en cada push a cualquier rama del repositorio. Su objetivo es validar la calidad del código antes de cualquier integración o despliegue:

- Análisis estático con ruff: verifica que el código sigue las convenciones de estilo de Python y detecta errores comunes sin ejecutar el código.
- Ejecución de tests con pytest: lanza todos los tests unitarios de la API, verificando el correcto funcionamiento de los tres endpoints.
- Control de cobertura: verifica que la cobertura de tests supera el umbral mínimo del 70%, garantizando que el código crítico está probado. El resultado obtenido es del 75%.

Job 2: build-and-push (Despliegue Continuo)

Este job solo se activa cuando hay un push a la rama principal y únicamente si el job de lint-and-test ha pasado correctamente. Esta dependencia explícita garantiza que nunca se construye ni despliega código que no ha superado las validaciones de calidad:

- Autenticación en ACR: el job se autentica en Azure Container Registry usando credenciales almacenadas como GitHub Secrets, nunca en el código fuente.
- Construcción de la imagen: ejecuta docker build con la etiqueta latest y el SHA del commit para trazabilidad.
- Push al registro: sube la imagen al ACR privado, desde donde Terraform la referencia para el despliegue en ACI.

[i] La separación entre CI y CD no es arbitraria: permite que los desarrolladores reciban feedback de calidad en cualquier rama sin activar despliegues innecesarios. Solo el código que ha pasado todas las validaciones y ha sido integrado en la rama principal llega a producción.

4.4 Infraestructura como Código con Terraform

Terraform permite definir la infraestructura Azure de forma declarativa: en lugar de configurar recursos manualmente en el portal de Azure, se describe el estado deseado en archivos .tf y Terraform se encarga de crear, modificar o eliminar los recursos necesarios para alcanzarlo.

La configuración de Terraform del proyecto está organizada en tres archivos:

- `main.tf`: define los recursos Azure a provisionar (Resource Group y Container Instance) y la referencia a la imagen Docker en ACR.
- `variables.tf`: declara las variables de configuración con sus valores por defecto, permitiendo reutilizar la misma configuración en distintos entornos.
- `outputs.tf`: expone la URL pública de la API, el nombre del Resource Group y la región de despliegue una vez completado el `terraform apply`.

5. Infraestructura y Despliegue en Azure

5.1 Recursos desplegados

La infraestructura del proyecto se compone de dos recursos principales en Azure, ambos definidos en código Terraform y desplegados en la región Spain Central:

Recurso Azure	Nombre	Propósito
Resource Group	rg-abc-analytics	Contenedor lógico que agrupa todos los recursos del proyecto y facilita su gestión y facturación conjunta
Azure Container Registry	ACR privado	Registro privado de imágenes Docker; recibe las imágenes construidas por el pipeline CI/CD
Azure Container Instance	sales-prediction-api	Instancia de contenedor que ejecuta la imagen Docker de la API con IP pública en el puerto 8000

5.2 Configuración del Container Instance

El recurso de Azure Container Instance está configurado con las siguientes especificaciones:

- Sistema operativo: Linux
- CPU: 0.5 vCPU (suficiente para una API de predicción con carga moderada)
- Memoria: 1.5 GB RAM
- Puerto expuesto: 8000 (TCP)
- Política de reinicio: Always (el contenedor se reinicia automáticamente si falla)
- Etiqueta DNS: abc-analytics-api (genera el FQDN abc-analytics-api.spaincentral.azurecontainer.io)
- Variables de entorno: APP_ENV=production

5.3 Proceso de despliegue con Terraform

El proceso de despliegue con Terraform sigue tres pasos estándar:

terraform init: descarga los providers necesarios (azurerm ~> 3.0) y configura el backend de estado.

terraform plan: analiza el estado actual de la infraestructura y genera un plan de cambios antes de aplicarlos. Permite revisar qué recursos se van a crear, modificar o eliminar.

terraform apply: aplica el plan de cambios, creando o actualizando los recursos en Azure. El tiempo total de aprovisionamiento es de aproximadamente 68 segundos.

[i] La idempotencia de Terraform es una de sus características más valiosas: si se ejecuta terraform apply sobre una infraestructura ya existente sin cambios en los archivos .tf, Terraform detecta que el estado real coincide con el deseado y no realiza ninguna modificación. Esto hace que el proceso de despliegue sea seguro de ejecutar repetidamente.

5.4 Gestión de credenciales

La seguridad en la gestión de credenciales es un aspecto crítico del proyecto. Las credenciales del ACR (login server, username y password) no aparecen en ningún archivo del repositorio. En su lugar, se gestionan mediante dos mecanismos:

- GitHub Secrets: las credenciales del ACR y la suscripción Azure se almacenan como secrets cifrados en el repositorio de GitHub. El pipeline CI/CD los inyecta como variables de entorno en tiempo de ejecución.
- Variables sensibles de Terraform: la variable acr_password está marcada como sensitive: true en variables.tf, lo que evita que Terraform la muestre en los logs de salida.

6. Metodología Ágil (Scrum)

6.1 Organización del trabajo

El proyecto se ha gestionado con la metodología Scrum, utilizando GitHub Projects como herramienta de gestión del tablero Kanban. El trabajo se ha organizado en tres sprints de dos semanas, con un backlog inicial de ocho User Stories priorizadas. El tablero de GitHub Projects está configurado con cinco columnas que reflejan el flujo de trabajo: Backlog, Ready, In Progress, In Review y Done. Cada User Story sigue el formato estándar de Scrum (Como [rol], quiero [acción] para [beneficio]) e incluye criterios de aceptación verificables.

URL tablero Kanban: <https://github.com/users/monicapenacho/projects/3>.

6.2 Sprints del proyecto

Sprint	Período	Objetivo	User Stories	Puntos	Estado
Sprint 1	Semana 1-2	API FastAPI + Docker + Tests unitarios	US-01, US-02, US-03	13/13	 Completado
Sprint 2	Semana 3-4	Pipeline CI/CD + Cobertura $\geq 70\%$	US-04, US-05, US-06	13/13	 Completado
Sprint 3	Semana 5-6	Terraform + Despliegue Azure + Documentación	US-07, US-08	6/16	 En curso

6.3 User Stories principales

ID	User Story	Criterios de Aceptación	Prioridad
US-01	Como desarrollador, quiero una API FastAPI con endpoint /predict para proporcionar predicciones de ventas	API arranca; /predict devuelve JSON con predicted_sales y confidence	P0
US-02	Como desarrollador, quiero containerizar la API con Docker para garantizar reproducibilidad del entorno	docker build y docker run funcionan; API accesible en localhost:8000	P0
US-03	Como desarrollador, quiero tests unitarios con cobertura $\geq 70\%$ para validar la API	pytest ejecuta 7 tests; cobertura $\geq 70\%$; todos en verde	P0
US-04	Como equipo, queremos un pipeline CI/CD automatizado para detectar errores antes de producción	Pipeline verde en cada push; jobs CI y CD diferenciados	P1
US-05	Como equipo, queremos publicar la imagen Docker en ACR para tener un registro privado	Imagen disponible en ACR tras cada push a main	P1
US-06	Como equipo, queremos definir la infraestructura con Terraform para reproducir el entorno en minutos	terraform apply crea Resource Group y ACI en < 90 segundos	P1

ID	User Story	Criterios de Aceptación	Prioridad
US-07	Como equipo, queremos desplegar la API en Azure ACI para que sea accesible públicamente	API accesible vía FQDN público; /health devuelve 200	P1
US-08	Como equipo, queremos documentar el proyecto con README y memoria para facilitar la transferencia de conocimiento	README completo; memoria ≥ 10 páginas; arquitectura documentada. US-08 en curso, prevista cierre Sprint 3	P2

6.4 Retrospectiva del proyecto

Al cierre del proyecto, el equipo identificó los siguientes aprendizajes clave:

Qué funcionó bien:

- La separación en dos jobs de GitHub Actions (CI y CD) permitió detectar errores de código sin activar despliegues innecesarios.
- El uso de variables sensibles de Terraform (sensitive: true) evitó la exposición accidental de credenciales en los logs.
- La documentación iterativa (README y memoria actualizados en cada sprint) redujo el esfuerzo de documentación final.

Qué mejoraríamos:

- Configurar los GitHub Secrets del ACR desde el Sprint 1, en lugar de hacerlo al llegar al Sprint 3.
- Añadir Branch Protection Rules desde el inicio para evitar pushes directos a main sin pasar por CI.
- Incluir un stage de staging entre CI y CD para validar la API en un entorno intermedio antes de producción.

7. Resultados Obtenidos

7.1 Métricas del proyecto

Métrica	Objetivo	Resultado	Estado
Cobertura de tests	≥ 70%	75%	✓
Tests unitarios	Todos en verde	7/7 pasados	✓
Pipeline CI/CD	Verde en cada push	Operativo y verde	✓
Tiempo terraform apply	< 120 segundos	~68 segundos	✓
Contenedor Azure	Running con IP pública	Desplegado en Spain Central	✓
Coste infraestructura	< 15€/mes	~8.50€/mes estimado	✓
Documentación Swagger	Disponible en /docs	Generada automáticamente por FastAPI	✓

⚠ Métricas medidas a fecha de presentación (6/05/2026); Sprint 3 sigue activo para cerrar US-08 (documentación de arquitectura cloud objetivo y Wave Planning). El resto de métricas técnicas son definitivas.

7.2 API desplegada en producción

La API de predicción de ventas está desplegada y accesible en la siguiente URL:

<http://abc-analytics-api-juanmonica.spaincentral.azurecontainer.io:8000/docs>

La documentación interactiva Swagger UI está disponible en la ruta /docs, permitiendo explorar y probar los endpoints directamente desde el navegador. Un ejemplo de llamada al endpoint /predict:

```
POST /predict
Content-Type: application/json

{
  "product": "ProductoA",
  "base_sales": 1000.0,
  "month": 5
}
```

Respuesta obtenida:

```
{
  "product": "ProductoA",
  "predicted_sales": 1187.43,
  "confidence": 0.87
}
```

8. Problemas Encontrados y Soluciones

El desarrollo del proyecto no estuvo exento de dificultades técnicas. A continuación, se documentan los problemas más relevantes junto con las soluciones aplicadas. Esta documentación forma parte del aprendizaje del equipo y demuestra que el proceso de desarrollo fue real y no una simulación.

Problema	Causa	Solución aplicada	Lección aprendida
AWS Educate no proporciona acceso a infraestructura real	La plataforma AWS Educate solo da acceso a cursos interactivos, no a servicios reales como EC2, S3 o ECR	Migración a Microsoft Azure Students, que ofrece 100€ de crédito real con el correo universitario y sin tarjeta de crédito	Verificar antes del Sprint 1 si la plataforma elegida soporta los requisitos técnicos del proyecto
Terraform devuelve error de permisos en el provider azurearm	Azure Cloud Shell tiene restricciones sobre el registro de providers de Azure por política de la suscripción	Añadir <code>skip_provider_registration = true</code> en el bloque <code>provider "azurerm"</code> del <code>main.tf</code>	Leer la documentación de Terraform para Azure en entornos de estudiante antes de comenzar
Conflicto de nombre DNS al ejecutar terraform apply	Azure requiere nombres DNS globalmente únicos para los Container Groups; el nombre inicial ya estaba en uso	Modificar el valor de <code>dns_name_label</code> en <code>main.tf</code> añadiendo un sufijo único. Verificación previa con <code>terraform plan</code>	Usar <code>terraform plan</code> antes de <code>apply</code> para detectar posibles conflictos sin modificar la infraestructura real
<code>ModuleNotFoundError</code> al ejecutar los tests en el pipeline CI/CD	Python no encontraba el módulo <code>app</code> porque el directorio raíz no estaba en el <code>PYTHONPATH</code> del entorno de CI	Añadir <code>__init__.py</code> en los directorios <code>app/</code> y <code>tests/</code> y configurar <code>PYTHONPATH</code> en el workflow de GitHub Actions	Probar el pipeline localmente simulando el entorno de CI antes del primer push
Pipeline de CI fallaba por archivos con caracteres especiales en rutas	El comando <code>ruff</code> analizaba notebooks (<code>.ipynb</code>) y archivos con tildes, generando errores de parseo	Acotar el análisis de <code>ruff</code> explícitamente a los directorios <code>app/</code> y <code>tests/</code> en el workflow	Definir el alcance de las herramientas de análisis estático desde el inicio del proyecto

9. Conclusiones

El proyecto de migración cloud de ABC Analytics ha demostrado de forma práctica cómo un conjunto de herramientas DevOps accesibles y de bajo coste puede transformar radicalmente la operación de una API, eliminando los procesos manuales y garantizando reproducibilidad y calidad en cada despliegue.

Conclusión 1: La IaC convierte la infraestructura en un activo reproducible. Gracias a Terraform, el entorno completo de Azure puede recrearse desde cero en aproximadamente 68 segundos con un único comando. Esta capacidad, impensable con configuración manual, tiene un valor crítico en escenarios de recuperación ante desastres o replicación de entornos.

Conclusión 2: El pipeline CI/CD es la garantía de calidad del ciclo de vida del software. La separación entre el job de CI (validación de calidad en todas las ramas) y el job de CD (despliegue solo en main y solo si CI pasa) asegura que ningún código defectuoso llega a producción de forma automática. Durante el desarrollo, el pipeline detectó y bloqueó varios errores que, sin automatización, habrían llegado al entorno de producción.

Conclusión 3: Docker elimina la dependencia del entorno de ejecución. La containerización garantiza que la API funciona de forma idéntica en el portátil del desarrollador, en el entorno de CI y en Azure Container Instances. Esto simplifica enormemente la resolución de problemas y la incorporación de nuevos miembros al equipo.

Conclusión 4: La metodología Scrum impuso una disciplina de entrega incremental. La organización en tres sprints permitió priorizar las funcionalidades críticas (API, Docker, CI/CD) antes de abordar el despliegue en la nube. Esta priorización resultó clave cuando surgieron impedimentos técnicos con la plataforma cloud, ya que el equipo disponía de una base funcional sólida.

Estado final del proyecto: El Sprint 3 está cerrando con la API ya desplegada en Azure y las últimas tareas de documentación en progreso (US-08: arquitectura cloud objetivo y Wave Planning). Los tres pilares técnicos —Docker, GitHub Actions y Terraform— están completamente operativos y en producción. El análisis 6R y el Wave Planning del portafolio completo están documentados en el notebook de portafolio del repositorio.

10. Trabajo Futuro

El proyecto actual constituye una base sólida sobre la que se pueden construir mejoras significativas en distintas dimensiones:

10.1 Observabilidad y monitorización

- Integración con Azure Monitor y Application Insights para centralizar logs, métricas de rendimiento y alertas. Actualmente la API genera logs estructurados que se pierden si el contenedor se reinicia.
- Implementación de trazas distribuidas con OpenTelemetry para rastrear el ciclo de vida completo de cada petición.
- Dashboard de métricas en Azure Portal con alertas automáticas cuando la latencia o el número de errores superan umbrales definidos.

10.2 Mejoras en el pipeline

- Añadir un entorno de staging entre CI y CD, donde la imagen Docker se despliega en un ACI de prueba y se ejecutan tests de integración antes de pasar a producción.
- Implementar Branch Protection Rules para exigir que el CI pase antes de permitir merge a main.
- Añadir pre-commit hooks que ejecuten ruff y los tests localmente antes de cada commit, reduciendo el tiempo de feedback.
- Integrar análisis de seguridad de la imagen Docker con herramientas como Trivy o Snyk para detectar vulnerabilidades en las dependencias.

10.3 Evolución del modelo de predicción

- Sustituir el modelo estocástico actual por un modelo de Machine Learning real (por ejemplo, Gradient Boosting o ARIMA) entrenado con datos históricos de ventas de ABC Analytics.
- Implementar un pipeline de MLOps completo con reentrenamiento automático periódico, registro de experimentos con MLflow y gate de calidad automático (el modelo solo se despliega si supera umbrales de AUC y F1 predefinidos).
- Añadir detección de data drift para identificar cuándo el modelo empieza a degradarse por cambios en los patrones de ventas.

10.4 Escalabilidad y alta disponibilidad

- Migrar de Azure Container Instances a Azure Kubernetes Service (AKS) para soportar auto escalado horizontal basado en carga.
- Implementar una estrategia de despliegue Canary o Blue/Green para actualizar la API sin tiempo de inactividad.
- Añadir una capa de caché (Azure Redis Cache) para respuestas frecuentes y reducir la latencia.

Esto es Wave 1. El plan completo tiene 3 oleadas — la siguiente es migrar el ETL nocturno como Rehost.

Referencias

- FastAPI — Documentación oficial: <https://fastapi.tiangolo.com>
- Terraform — Documentación azure provider: <https://registry.terraform.io/providers/hashicorp/azurerm>
- GitHub Actions — Documentación oficial: <https://docs.github.com/en/actions>
- Azure Container Instances — Documentación: <https://learn.microsoft.com/en-us/azure/container-instances/>
- Azure Container Registry — Documentación: <https://learn.microsoft.com/en-us/azure/container-registry/>
- Docker — Documentación oficial: <https://docs.docker.com>
- Repositorio del proyecto: <https://github.com/jmcampos-sec/sales-prediction-api-migration>

Juan Manuel Campos Enrique / Mónica Penacho · 20GIAR · VIU · Mayo 2026